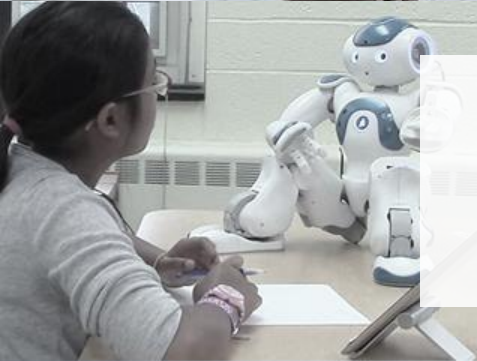
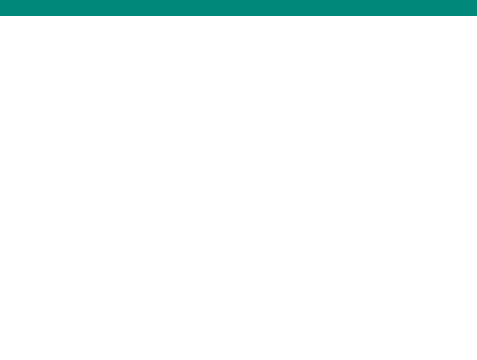




Lab 6: Gradient Descent Inverse Kinematics for Pick and Place

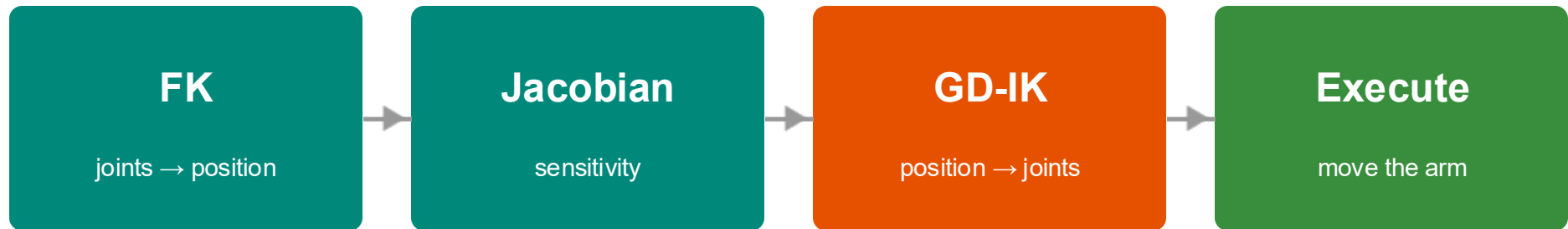
Overview

Pick up **red block** from table → drop in **green bin**

You implement:

1. Forward Kinematics
joints → gripper position
2. Numerical Jacobian
how each joint affects the gripper
3. Gradient Descent IK
desired position → joints
4. Pick-and-Place Sequence
put it all together

The Big Picture



Each step builds on the previous one.

FK is the foundation, everything else depends on it.

Task 1: Forward Kinematics (15 pts)

DH convention gives you a 4×4 transform per joint
(textbook Eq. 3.9, Chapter 3.1.2)

Implementation:

$T = I$ (T is the 4x4 homogenous transformation matrix)
for each joint: $T = T \times \text{DH}(a, d, \alpha, \theta)$
add gripper offset along tool Z
return position = $T[:3, 3]$

DH parameters + `_dh()` function are prov

Variables

T: 4×4 transform matrix

a: link length (m)

d: link offset (m)

α : link twist (rad)

θ : joint angle (rad)

Gripper offset: Robotiq gripper length)

$T[:3, 3]$ — [x,y,z] position

$${}_{n-1}^nT = \left[\begin{array}{ccc|c} \cos \theta_n & -\sin \theta_n \cos \alpha_n & \sin \theta_n \sin \alpha_n & r_n \cos \theta_n \\ \sin \theta_n & \cos \theta_n \cos \alpha_n & -\cos \theta_n \sin \alpha_n & r_n \sin \theta_n \\ 0 & \sin \alpha_n & \cos \alpha_n & d_n \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

Task 2: Numerical Jacobian (15 pts)

In lecture 6 (slide 12): J maps joint changes to position changes

$$\Delta \text{position} = \mathbf{J} \cdot \Delta \text{joints}$$

You compute J numerically, wiggle each joint, measure FK change:

$$\mathbf{J}[:, i] = (\text{FK}(\mathbf{q} + \delta) - \text{FK}(\mathbf{q} - \delta)) / (2\delta)$$

Result: 3×6 matrix (3 positions × 6 joints)

Uses your FK from Task 1 — that's it!

J: 3×6 matrix. Column i = how gripper [x,y,z] changes when joint i moves.

δ: tiny perturbation (1e-5 rad). **e_i**: unit vector for joint i (perturb only that joint).

FK(): your forward kinematics from Task 1.

Task 4: Gradient Descent IK (20 pts)

Recall: analytical IK is hard (slide 8). We use gradient descent instead.

The algorithm:

$\text{error} = \text{target} - \text{FK}(\mathbf{q})$

$\mathbf{J} = \text{compute_jacobian}(\mathbf{q})$

$\mathbf{q} = \mathbf{q} + \alpha \cdot \mathbf{J}^T \cdot \text{error}$

repeat until $\|\text{error}\| < 5\text{mm}$

Why $\mathbf{J}^T$ not $\mathbf{J}^{-1}$? Simpler, always defined, no singularity issues.
(Lecture slide 13 shows $\mathbf{J}^{-1}$ — we simplify to $\mathbf{J}^T$ for this lab)

Variables

$\mathbf{q}$: 6×1 joint angles (rad)

Target: desired [x,y,z] in base frame

Error: 3×1 vector, where to where

α : learning rate (default 0.5)

$\mathbf{J}^T$: 6×3 transpose of Jacobian

$\mathbf{J}^T \cdot \text{error}$: 6×1 direction in joint space that reduces error

$\|\text{error}\|$: $\sqrt{e_x^2 + e_y^2 + e_z^2}$

Tasks 3 & 5: Waypoints + Pick & Place



Task 3: Define `above_block`, `at_block`, `above_bin` positions

Task 5: Use your IK to get joint angles, then execute the sequence

Gripper interface provided: `open_gripper()`, `close_gripper()`

Report (30 pts)

FK Verification

10 pts

Test FK at 3 configs.
Verify in Webots.
Explain GRIPPER_OFFSET.

IK Analysis

10 pts

3 targets, report convergence.
Sweep learning rates:
0.1 vs 0.5 vs 2.0
Plot error vs iteration.

Reflection

10 pts

Role of Jacobian in IK.
Why J^T not J^{-1} ?
Limitations?
Screenshot of result.

Tips & Getting Started

Build incrementally: FK first, then Jacobian, then IK, then sequence

Test each function: `main()` runs diagnostics automatically

If IK doesn't converge: check `world_to_base()` conversion

Coordinate frames: `world_to_base()` and `base_to_world()` are provided

References: Textbook Ch. 3.1.2 (DH), Lecture 6 slides 8–13 (IK + Jacobian)

Due: April 7th on Canvas